academic-pdf-reading

Installation and setup guide

Skill version 1.2.0 · 2 October 2026

What this skill does

academic-pdf-reading is a Claude skill for reading academic papers and textbooks without transcription errors. Text extracted from scientific documents fails silently: a superscript -1 becomes a subtraction, an overbar vanishes, a prime turns into the digit 0. The skill enforces one rule — *text layers are for locating and prose; mathematics is read from page images* — and supplies the procedures and a measurement script that make the rule practical. It also maintains a small per-project file recording verified facts about that project's documents.

New in v1.2.0: renditions

The same uploaded document can be mounted in different forms in different sessions — for example as the original PDF in one session and as a ZIP of page images plus text layers in another — without you changing anything. Earlier versions reported every such switch as a changed file. From v1.2.0 the inventory records several *renditions* per file, the staleness check recognises any recorded one, and an unrecorded form of an apparently unchanged document is reported as a *probable rendition swap* that Claude confirms with you instead of a full re-triage (Sections 5 and 6.2).

Scope of this guide

This guide covers installing the skill, setting up a project, and keeping the setup current. Menu names and locations in the Claude app change over time; where this guide refers to a setting, check `https://support.claude.com` for its current location.

Contents

1 The components 2

2 Requirements 2

3 Installing or updating the skill 3

3.1 Option A (recommended): upload the skill ZIP . . . 3

3.2 Option B: use the Markdown bundle without installing . . . 3

4 Setting up a project 4

4.1 Generate the project protocol . . . 4

4.2 Upload it . . . 4

4.3 Point the project instructions at it . . . 4

4.4 Confirm the read-back . . . 4

5 Day-to-day use 5

6 Updating the protocol **5**
6.1 After a stale result 5
6.2 After a confirmed rendition swap 5
7 Updating the skill itself **6**
8 Troubleshooting **6**
A Command reference: `triage.py` **7**

1 The components

There are three deliverables. Only the first is required.

File	Purpose
`academic-pdf-reading-v1.2.0.zip (or .skill)`	The installable skill: a ZIP of the skill folder. A `.skill` file is the same ZIP under another extension. Install once; available in every chat and project.
`academic-pdf-reading.md`	The same skill as one Markdown file, for use *without* installing (upload into a project's knowledge). Generated from the skill; never edit it by hand.
`pdf_protocol_<project>.md`	One per project: measured facts about that project's files, plus a machine-readable inventory. Generated by Claude with the skill, uploaded by you.

Inside the skill:

```
academic-pdf-reading/
  SKILL.md                          policy, triage, renditions, access
                                    recipes, failure modes, quoting rules
  scripts/triage.py                 deterministic measurements
  scripts/build_bundle.py           regenerates the .md bundle
  references/project_protocol.md    how to create/update a project protocol
```

2 Requirements

- **Skills enabled** for your account. On Team and Enterprise plans an administrator may need to allow them.
- **Code execution and file creation enabled**, since the skill runs `triage.py`. Without it the skill's text still applies, but no measurements, crops or staleness checks can run.
- Nothing to install yourself. The script needs Python 3.8+ and uses poppler-utils and Pillow when present; Claude's sandbox provides them, and the script degrades gracefully where a tool is missing (`triage.py tools` reports what exists).

3 Installing or updating the skill

3.1 Option A (recommended): upload the skill ZIP

First installation.

1. Download the skill ZIP (or `.skill`) file.
2. Install it. When Claude presents a `.skill` file in chat, its file card normally offers a **Save skill** button. Otherwise go to **Customize** → **Skills**, click **+**, choose **Create skill**, then **Upload a skill**, and select the ZIP.
3. Make sure the skill is toggled on.

Updating an installed version. The skill keeps the same name (`academic-pdf-reading`) across versions.

1. In **Customize** → **Skills**, open the existing skill's three-dot menu and look for an **Upload and replace** option. If it exists, use it with the new ZIP.
2. If it does not, delete the old skill and upload the new ZIP as for a first installation. Nothing is lost: the ZIP contains the complete skill, and project protocols live in project knowledge, not in the skill.
3. Confirm that exactly one copy is listed and enabled. Two copies with the same name and description compete for the same requests.

If the skill is not listed under Skills

A skill can also arrive through a plugin rather than a direct upload; it is then not necessarily editable from the Skills list. Update it the way it was installed, and ask Claude where the skill is mounted if you are unsure (`/mnt/skills/user/` for uploaded skills; other paths for others).

Verify. In a new chat, ask:

```
Which version of the academic-pdf-reading skill is installed? Print
metadata.version from SKILL.md and TRIAGE_VERSION from triage.py.
```

Both should read `1.2.0`. To check that the skill works, attach any paper and ask Claude to triage it with the skill: it should run `triage.py profile` and report the file's container (genuine PDF, ZIP archive, or text extraction), its channels, and any text-layer warnings.

3.2 Option B: use the Markdown bundle without installing

Use this where skills are unavailable, or to pin a version in one project.

1. Upload `academic-pdf-reading.md` to the project's knowledge.
2. In each session Claude extracts the script from Part 3 of the bundle. The bundle's opening snippet does this:

```
import re
t = open('/mnt/project/academic-pdf-reading.md', encoding='utf-8').read()
s = re.search(r'~~~~python\n(.*?)\n~~~~', t, re.S).group(1)
open('/home/claude/triage.py', 'w', encoding='utf-8').write(s)
```

3. Tell Claude to follow the bundle (see the pointer text in Section 4.3, replacing "skill" with "`academic-pdf-reading.md` in project knowledge").

Caution

If the bundle is uploaded into a project that also has a protocol file, the staleness check treats the bundle as a source file and does *not* flag it (from v1.1.0). In v1.0.0 it was flagged as an added file.

4 Setting up a project

Do this once per project whose documents you will ask about repeatedly. For a one-off paper, skip it: the skill triages files on the fly.

4.1 Generate the project protocol

Upload the project's papers first. Then ask:

```
Generate a project-specific PDF protocol for this project, following the
academic-pdf-reading skill's references/project_protocol.md.
```

Claude profiles every file, verifies glyph mappings and page offsets on the page images, and writes `pdf_protocol_<project>.md` with an embedded inventory: for each file, the rendition currently mounted (size, SHA-256 hash, container, page count, and a prose sketch used to recognise the same document in another form later). This is the most token-intensive step, and it happens once. Other renditions are added as they appear (Section 6.2); there is no need to wait for uploads to "settle" first.

4.2 Upload it

Claude cannot write into project knowledge, so download the file Claude presents and upload it to the project's knowledge yourself.

4.3 Point the project instructions at it

Add this to the project's instructions (custom instructions). The last sentence keeps the core rule in force even in a session where the skill fails to load:

> PDF reading: follow the academic-pdf-reading skill. Project-specific facts are in `pdf_protocol_<project>.md` in project knowledge; run its staleness check on first use each session. Invariant even without the skill: text layers are for locating and prose only; equations, signs, indices, bars and primes are read from page images, and math from text-only files is labelled unverified.

If the instructions previously contained a full reading protocol, remove it *after* the next step succeeds; keeping both copies leads to drift.

4.4 Confirm the read-back

In a *new* chat in the project, ask:

```
Run the PDF protocol staleness check.
```

Expected result: the skill triggers, finds the protocol, and reports **up to date**. Only then delete any old protocol text from the instructions.

5 Day-to-day use

The staleness check runs once per session at the first use of the skill, and again if you say files changed — not per question. It has three outcomes:

Result	Exit	What Claude does
up to date	0	Uses the protocol's facts for the rendition that is mounted (reported as `mounted` lines when a file has several).
`SWAP?` only (probable rendition swap)	3	Lists the files and asks whether any was edited or replaced. Until you answer, reads them without the protocol. If you confirm nothing changed, offers to record the new renditions (Section 6.2).
STALE	1	Lists what was added, removed or changed, reads those files without the protocol, and offers an update.

No outcome blocks an answer.

- **When a file counts as a probable swap.** Its hash matches no recorded rendition, but its container differs from every recorded one, its page count matches, and its prose sketch (where one is recorded) agrees with the recorded document. Anything else is reported as `CHANGED`, including a new hash in an *already recorded* container, which may be a revision.
- **Why your answer is needed.** The sketch identifies the *document*, not the *version*: a revision that only changes a few equations or sentences looks identical to it. Only you know whether a file was edited. If you are unsure, say so, and Claude treats the file as changed.
- **Source files do not trigger updates.** Adding, editing or removing `.tex`, `.md`, `.bib`, `.sty` or `.cls` files is reported as `source +/-/~` and the result stays up to date, so frequent chat-summary uploads cause no update rounds. A `.txt` file is *not* treated as source: it may be a text extraction of a book.

Keeping token use low. Ask about specific equations or pages ("Kugler Eq. (46)") rather than whole sections; start a fresh chat per topic, since images accumulate in context; say when a text-layer reading is good enough.

6 Updating the protocol

6.1 After a stale result

When you accept an update, Claude re-triages only the affected files. A changed file is treated as new: its old renditions and every fact recorded for them are retired. Claude then regenerates the inventory (keeping the recorded renditions of all other files), adds a changelog line and hands you the new file to re-upload.

6.2 After a confirmed rendition swap

Once you confirm that the files were not edited, Claude:

1. triages the newly mounted rendition and records its container-dependent facts (access recipe, glyph mappings, line endings, image geometry) in a separate block, leaving the facts of the existing rendition untouched;
2. confirms the document-level facts (page offset, equation-tag range, structure) on a few pages of the new rendition;
3. appends the new rendition to the inventory for exactly the files you confirmed, and hands you the protocol to re-upload.

From then on, either rendition passes the check silently, and Claude uses whichever rendition's facts match the file that is mounted. Mappings are never carried from one rendition to another: a text layer extracted from a ZIP and one extracted from the original PDF damage mathematics differently.

7 Updating the skill itself

The skill folder is the single source of truth.

1. Edit files in the skill folder (or ask Claude to; Claude works on a copy, since installed skills are read-only).
2. In `SKILL.md`, bump `metadata.version` and `metadata.date`; also bump `TRIAGE_VERSION` in `triage.py` if the script changed.
3. Rebuild the bundle and repackage:

   ```
   python3 academic-pdf-reading/scripts/build_bundle.py academic-pdf-reading.md
   python3 -m scripts.package_skill academic-pdf-reading OUTDIR   # skill-creator
   ```

   A plain ZIP of the folder also works:

   ```
   zip -r academic-pdf-reading-<version>.zip academic-pdf-reading
   ```

4. Reinstall the skill (Section 3.1); re-upload the bundle wherever it is used. The bundle's first line carries the version, so a mismatch with the installed skill is visible.

Protocol compatibility. v1.2.0 introduces inventory format 2 (several renditions per file). It reads format 1 inventories (v1.0.0 and v1.1.0), each file becoming a single rendition, so existing protocols keep working without regeneration. The next protocol update writes format 2.

8 Troubleshooting

The skill does not trigger.
Name it explicitly ("use the academic-pdf-reading skill"). Check that it is installed and enabled, and that code execution is enabled.

The check reports `SWAP?` although I changed nothing.
Expected after a rendition switch: the platform mounted a form of the file that the protocol has not recorded yet. Confirm that nothing changed and accept the offered update; afterwards both forms pass. Switches have been observed in both directions (PDF → ZIP and ZIP → PDF), so waiting does not prevent them; recording both renditions does.

The check reports `CHANGED` although I changed nothing.
The file's form was not recognisable as the same document: a different page count, a disagreeing

sketch, or a new hash in an already recorded container (for example, the platform re-rendered a ZIP). Tell Claude nothing changed; Claude still re-triages the file, because a recorded rendition's facts cannot be assumed to hold for a new rendering.

A `.pdf` is not a PDF.
Expected. Project files may be ZIP archives (page images plus text layers) or plain-text extractions with a `.pdf` extension. The skill identifies files by their first bytes, never by extension.

A textbook has no page images.
Text-only extractions have no image channel. Mathematics from them is labelled *text-layer reading, unverified* and should be cross-checked against a paper that does have images, or re-derived.

The bundle's script extraction fails.
The snippet assumes `/mnt/project/academic-pdf-reading.md`; change the path if the bundle lives elsewhere.

A tool is missing.
Run `triage.py tools`. Without a PDF renderer, genuine PDFs fall into the unverified regime until one is available; Claude will say so.

A Command reference: `triage.py`

Run from the skill directory as `python3 scripts/triage.py <command>`. Exit codes: 0 success or up to date; 1 stale; 2 usage or I/O error; 3 only probable rendition swaps.

Command	What it does
`tools`	Lists available helper tools.
`profile FILE... [--json OUT]`	Container, channels, text-layer signals, page-offset candidates, notes. Measures; never decides.
`inventory FILE... [--root DIR] [--out OUT]`	Inventory for a protocol: per file, its renditions (size, SHA-256, container, pages, prose sketch).
`[--merge OLD]`	Keeps OLD's recorded renditions for files whose hash is already known. A file with a new hash replaces its entry.
`[--add-rendition NAME...]`	With `--merge`: the named files' new hashes are appended as further renditions instead. Only for files the user confirmed unchanged.
`check INVENTORY [--root DIR] [--json]`	Staleness check against a `.json` or a protocol `.md`. A file matching any recorded rendition is unchanged. Exit 1 if stale, 3 if only probable swaps.
`text FILE [N] [--escape]`	One page of a text layer (or a line range of a text file); `--escape` shows control bytes.
`grep PATTERN FILE... [-i]`	Regex search reported as file:page:line; CRLF-safe.
`extract FILE --out DIR`	Unpacks a ZIP; splits a PDF into per-page `N.txt`.
`crop SRC --page N [--box x0 y0 x1 y1] --out PNG`	Page image, optionally cropped and upscaled, from a ZIP, PDF or image.